MongoDB 7.0 Aggregation Pipelines & Document Modeling

Pipeline stages, schema trade-offs, indexes, joins, and explain-driven optimization

1. Documents and schema design

MongoDB stores BSON documents in collections. Document modeling should follow the application's access patterns: what is commonly read together, how data changes, expected cardinality, and document growth. A flexible schema is not a reason to skip validation or consistency rules.

```
{
  _id: ObjectId("..."),
  customerId: ObjectId("..."),
  status: "paid",
  createdAt: ISODate("2026-01-12T10:00:00Z"),
  items: [
    { sku: "BK-101", quantity: 2, unitPrice: 14.50 }
  ]
}
```

Design notes: Embed bounded child data that is commonly read with its parent. Reference independently queried, high-cardinality, or frequently changing entities. Consider document-size limits, update contention, duplication, and whether atomic updates must cover multiple entities.

Practice: Sketch models for a shopping cart, a product catalog, and an audit log. Explain why the same embedding strategy should not be used for all three.

2. Pipeline mental model

An aggregation pipeline is an ordered sequence of stages. Each stage transforms the documents flowing through it, filtering, reshaping, grouping, sorting, joining, or computing values. The output of one stage becomes the input of the next.

```
db.orders.aggregate([
  { $match: { status: "paid" } },
  { $unwind: "$items" },
  { $group: {
      _id: "$items.sku",
      unitsSold: { $sum: "$items.quantity" },
      revenue: {
        $sum: { $multiply: ["$items.quantity", "$items.unitPrice"] }
      }
  }},
  { $sort: { revenue: -1 } },
  { $limit: 10 }
])
```

Design notes: Start with the business question and define the desired output shape. Pipeline order affects both correctness and performance. Some stages may be optimized or reordered by MongoDB, but the logical result must remain clear.

Practice: Trace a single order with two items through each stage and write down the shape after every transformation.

Additional review: Implementation lens: isolate the smallest reproducible example, state the expected result, and test it independently before integrating it into a larger application.

3. Filter early and project deliberately

$match selects documents satisfying a predicate. A leading $match can use collection indexes where eligible and reduces downstream work. $project controls output fields and can compute derived fields; it is often most useful when shaping the final result or making a pipeline's intent clear.

```
db.orders.aggregate([
  { $match: {
      status: "paid",
      createdAt: { $gte: ISODate("2026-01-01T00:00:00Z") }
  }},
  { $project: {
      customerId: 1,
      createdAt: 1,
      total: 1,
      _id: 0
  }}
])
```

Design notes: Don't project away fields needed by later stages. A leading $match is a strong default, but avoid rewriting pipelines based on simplistic rules alone; inspect explain output and account for optimizer behavior.

Practice: Compare result counts and execution statistics for a broad scan versus a selective date/status filter.

4. Grouping and accumulators

$group aggregates documents by a key. Common accumulators include $sum, $avg, $min, $max, $first, $last, and $push. $group does not guarantee output ordering; add $sort when order matters. Memory consumption and cardinality can dominate performance.

```
db.events.aggregate([
  { $match: { kind: "purchase" } },
  { $group: {
      _id: "$userId",
      purchaseCount: { $sum: 1 },
      totalValue: { $sum: "$amount" },
      averageValue: { $avg: "$amount" }
  }},
  { $sort: { totalValue: -1 } }
])
```

Design notes: Use $first and $last only when the input ordering is deliberate for the intended meaning. If a group key has almost as many values as the input, grouping may provide little reduction and can be costly.

Practice: Group events by day and event type. Define the timezone policy explicitly before truncating timestamps to dates.

5. Arrays and $unwind

$unwind emits one pipeline document per element of an array, which is useful for per-item analysis. It can multiply document counts dramatically. The preserveNullAndEmptyArrays option controls whether documents with missing, null, or empty arrays survive.

```
db.orders.aggregate([
  { $unwind: {
      path: "$items",
      preserveNullAndEmptyArrays: false
  }},
  { $group: {
      _id: "$items.category",
      itemCount: { $sum: 1 },
      units: { $sum: "$items.quantity" }
  }}
])
```

Design notes: Know whether you are counting orders, line items, or units sold; these are different metrics. If only some array elements are needed, consider filtering arrays before unwinding where appropriate. Test empty arrays and missing fields explicitly.

Practice: Given an order with three line items, calculate how many documents exist after $unwind and how each metric should be defined.

Additional review: Review lens: consider edge cases, failure behavior, maintainability, and the assumptions the code makes about its input or execution environment.

6. Joins with $lookup

$lookup performs a left outer join from the current collection to another collection. It can enrich documents with matching records, but large joins can be expensive. Index the foreign join field and return only the fields needed by the application.

```
db.orders.aggregate([
  { $match: { status: "paid" } },
  { $lookup: {
      from: "customers",
      localField: "customerId",
      foreignField: "_id",
      as: "customer"
  }},
  { $unwind: "$customer" },
  { $project: {
      orderNumber: 1,
      total: 1,
      customerName: "$customer.name"
  }}
])
```

Design notes: An array field is created by $lookup; $unwind changes the shape and can drop unmatched records unless configured to preserve them. Consider whether a reference, denormalized snapshot, or lookup best fits the access pattern.

Practice: Decide how the API should behave when an order references a customer that has been deleted or archived.

7. Computed fields and conditional logic

$set (also known historically as $addFields) adds or replaces fields. Expressions such as $cond, $ifNull, $multiply, and date operators can derive values without application-side loops. Keep currency arithmetic and missing-field semantics deliberate.

```
db.orders.aggregate([
  { $set: {
      subtotal: {
        $sum: {
          $map: {
            input: { $ifNull: ["$items", []] },
            as: "item",
            in: { $multiply: ["$$item.quantity", "$$item.unitPrice"] }
          }
        }
      }
  }},
  { $set: {
      orderBand: {
        $cond: [{ $gte: ["$subtotal", 100] }, "large", "standard"]
      }
  }}
])
```

Design notes: Be explicit about null, missing, and type-mismatched fields. For financial values, choose a consistent representation and rounding policy; Decimal128 is available when decimal arithmetic is required. Do not silently treat malformed data as zero unless that is the defined business rule.

Practice: Test documents with missing items, zero quantity, null prices, and a total exactly on the threshold.

8. Embedded versus referenced data

Embedding can make reads efficient and allows atomic updates to a single document, but duplicates data and can grow documents. Referencing reduces duplication and supports independently managed entities, but may require additional queries or $lookup. Choose based on relationship cardinality and access patterns, not a blanket rule.

```
// Embedded snapshot: useful when the order must preserve purchase-time details
{
  orderId: "O-104",
  items: [
    { productId: "P-9", nameAtPurchase: "Notebook",
      quantity: 2, unitPrice: Decimal128("4.25") }
  ]
}

// Reference: useful for a separately managed product catalog
{
  _id: "P-9",
  name: "Notebook",
  currentPrice: Decimal128("4.75")
}
```

Design notes: Order line items often retain a purchase-time price/name snapshot even if a product's current catalog data changes. Conversely, a user profile should generally not be copied into every unrelated event unless a deliberate immutable snapshot is needed.

Practice: Write down read frequency, update frequency, maximum child count, and consistency requirements before choosing embedding or references.

Additional review: Optimization lens: measure behavior with representative data and workload; check the relevant official documentation before relying on implementation-specific behavior.

9. Indexes and explain-driven optimization

Indexes can speed up filtering and sorting, but they consume storage and increase write cost. Compound index key order matters, particularly for equality filters, sort requirements, and range conditions. Use explain('executionStats') to compare examined keys/documents with returned results and identify scans or expensive stages.

```
db.orders.createIndex({ status: 1, createdAt: -1 });

db.orders.explain("executionStats").aggregate([
  { $match: { status: "paid" } },
  { $sort: { createdAt: -1 } },
  { $limit: 50 }
])
```

Design notes: An index is not automatically useful merely because a field appears in a query. Check selectivity, sort compatibility, index prefixes, and actual workload. Explain plans are evidence for a particular query and data distribution, not a permanent guarantee.

Practice: Compare totalKeysExamined, totalDocsExamined, nReturned, and execution time before and after a candidate index.

10. Pipeline correctness, limits, and production patterns

Production aggregation needs bounded work, explicit output contracts, realistic data-volume testing, and monitoring. $sort and $group can be resource-intensive; some stages have memory constraints and disk-use behavior. Avoid returning huge result sets when pagination or pre-aggregation would satisfy the product requirement.

```
db.orders.aggregate([
  { $match: { status: "paid" } },
  { $group: {
      _id: {
        year: { $year: "$createdAt" },
        month: { $month: "$createdAt" }
      },
      orders: { $sum: 1 },
      revenue: { $sum: "$total" }
  }},
  { $sort: { "_id.year": 1, "_id.month": 1 } }
], { allowDiskUse: true })
```

Design notes: allowDiskUse is not a replacement for a sensible pipeline or indexes. Check the exact stage limits and server settings for the MongoDB version deployed. Use representative production-like distributions and monitor slow operations.

Practice: Capstone: build a monthly sales report with date filtering, grouping, a customer lookup, and an explain-based index review.

Trusted documentation and further reading

- **MongoDB 7.0 — Aggregation pipeline** — https://www.mongodb.com/docs/v7.0/core/aggregation-pipeline/
- **MongoDB 7.0 — Pipeline stages** — https://www.mongodb.com/docs/v7.0/reference/operator/aggregation-pipeline/
- **MongoDB 7.0 — Data modeling** — https://www.mongodb.com/docs/v7.0/data-modeling/
- **MongoDB 7.0 — Embedded data models** — https://www.mongodb.com/docs/v7.0/data-modeling/embedding/
- **MongoDB 7.0 — Referenced data models** — https://www.mongodb.com/docs/v7.0/data-modeling/referencing/
- **MongoDB 7.0 — $lookup** — https://www.mongodb.com/docs/v7.0/reference/operator/aggregation/lookup/
- **MongoDB 7.0 — Aggregation pipeline optimization** — https://www.mongodb.com/docs/v7.0/core/aggregation-pipeline-optimization/
- **MongoDB 7.0 — Explain** — https://www.mongodb.com/docs/v7.0/reference/method/db.collection.explain/

Subject review checklist

- Schema follows query and update patterns.
- Pipeline stages preserve the intended document shape.
- Array expansion and grouping cardinality are understood.
- Indexes are justified by explain output.
- Resource use and result sizes are bounded.